End-to-End Healthcare Project

1. Project Overview

This project implements an **end-to-end healthcare data engineering pipeline** on AWS that ingests healthcare staffing datasets from Google Drive, processes them using AWS Glue, stores curated datasets in a structured data lake, and enables analytical reporting through SQL views and dashboards.

The pipeline automates ingestion, transformation, and analytics to provide operational insights into healthcare staffing efficiency and hospital utilization.

Key metrics generated include:

- Nurse-to-patient ratios
- Total nurse staffing hours
- Bed utilization rates
- Facility occupancy trends
- Staffing vs patient load analysis

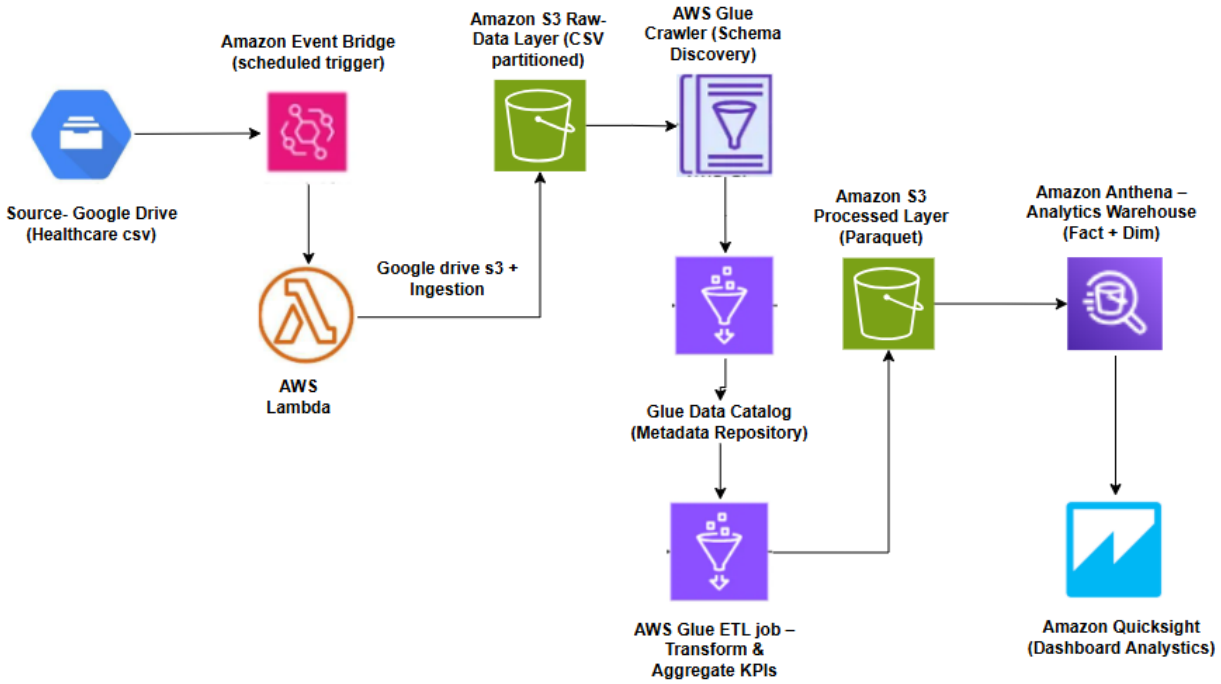
The final dataset powers an **interactive dashboard for healthcare operational analytics.**

2. Architecture Overview

The pipeline follows a **modern cloud data lake architecture**.

```
Google Drive (Source Files)
        |
        ▼
AWS Lambda (Ingestion)
        |
        ▼
Amazon S3 Raw Layer
        |
        ▼
AWS Glue Crawler
        |
        ▼
Glue Data Catalog
        |
        ▼
AWS Glue ETL (PySpark)
        |
        ▼
Amazon S3 Processed Layer (Parquet)
        |
        ▼
Glue Data Catalog Tables
        |
```

```
        ▼
Athena SQL Views
        |
        ▼
Amazon QuickSight Dashboard
```



The pipeline is **schedule-driven using Amazon EventBridge** and includes **monitoring and alerting mechanisms** using CloudWatch and SNS.

3. Source Data

Healthcare datasets are stored as CSV files in Google Drive.

Source Files

File	Description
PBJ_Daily_Nurse_Staffing_Q2_2024.csv	Nurse staffing data
NH_ProviderInfo_Oct2024.csv	Hospital/provider information
NH_QualityMsr_Claims_Oct2024.csv	Healthcare quality metrics

File	Description
PBJ_Daily_Nurse_Staffing_Q2_2024.csv	Nurse staffing data
NH_ProviderInfo_Oct2024.csv	Hospital/provider information
NH_QualityMsr_Claims_Oct2024.csv	Healthcare quality metrics

Google Drive Folder

```
Healthcare_datasets
```

This folder acts as the **external source system for the ingestion pipeline.**

4. Google Cloud Configuration

To allow AWS services to securely access Google Drive, a **Google Cloud project and service account** were created.

Steps

1. Create a project in **Google Cloud Console**
2. Enable **Google Drive API**
3. Create a **Service Account**
4. Generate **Service Account JSON key**
5. Download credentials

The service account enables **programmatic access to Google Drive from AWS Lambda.**

5. Grant Google Drive Access

The Google Drive folder must be shared with the service account.

Steps

1. Open Google Drive
2. Navigate to:

```
Healthcare_datasets
```

3. Click **Share**
4. Add the **Service Account Email**
5. Grant **Viewer access**

This allows AWS Lambda to download the source files.

6. Secure Credential Storage

The Google service account credentials are stored securely using **AWS Secrets Manager**.

Secret Name

```
gdrive_service_account_json
```

Why Secrets Manager

- prevents credential exposure
- avoids hardcoding credentials in Lambda
- supports secure credential rotation

7. Amazon S3 Data Lake Structure

Amazon S3 is used as the **data lake storage layer**.

Bucket

```
healthcare-metrics-hmanthena
```

Folder Structure

```
healthcare-metrics-hmanthena
│
├── raw
│   ├── pbj
│   │   └── load_date=2026-03-03
│   │       └── PBJ_Daily_Nurse_Staffing_Q2_2024.csv
│   │
│   ├── provider
│   │   └── load_date=2026-03-03
│   │       └── NH_ProviderInfo_Oct2024.csv
│   │
│   └── quality
│       └── load_date=2026-03-03
│           └── NH_QualityMsr_Claims_Oct2024.csv
│
├── processed
│
└── logs
```

8. Raw Data Partitioning Strategy

Files are stored using **load_date partitions**.

Example:

```
raw/pbj/load_date=2026-03-03/PBJ_Daily_Nurse_Staffing_Q2_2024.csv
raw/provider/load_date=2026-03-03/NH_ProviderInfo_Oct2024.csv
raw/quality/load_date=2026-03-03/NH_QualityMsr_Claims_Oct2024.csv
```

Benefits of Partitioning

- Supports **incremental data ingestion**
- Enables **historical data retention**
- Improves **Athena query performance**
- Helps **debug ingestion issues**
- Provides **data lineage and auditing**

9. AWS Lambda Ingestion Pipeline

An AWS Lambda function is used to **automatically ingest data from Google Drive into S3.**

Lambda Responsibilities

The Lambda function performs the following steps:

1. Retrieve Google Drive credentials from **Secrets Manager**
2. Authenticate using the service account
3. Connect to the **Healthcare_datasets** folder
4. Identify required CSV files
5. Download the files
6. Upload files to the S3 raw layer

File Mapping

Source File	Destination
PBJ_Daily_Nurse_Staffing_Q2_2024.csv	raw/pbj
NH_ProviderInfo_Oct2024.csv	raw/provider
NH_QualityMsr_Claims_Oct2024.csv	raw/quality

10. Lambda IAM Permissions

The Lambda execution role includes permissions for:

Permission	Purpose
secretsmanager:GetSecretValue	Access credentials
s3:PutObject	Upload files
s3:ListBucket	Check S3 bucket
glue:StartJobRun	Trigger Glue ETL
CloudWatch Logs	Logging

11. Automated Scheduling Using EventBridge

The ingestion pipeline is **schedule-driven using Amazon EventBridge**.

Schedule

Runs twice per day:

- **5:00 AM**
- **5:00 PM**

Cron Expression

```
cron(0 5,17 * * ? *)
```

Execution Flow

```
EventBridge
     |
     ▼
Lambda Function
     |
     ▼
S3 Raw Layer
     |
     ▼
Glue ETL Job
```

This ensures that **data ingestion is fully automated.**

Steps-

To automate the ingestion pipeline, **Amazon EventBridge** was configured to trigger the Lambda ingestion function on a scheduled basis.

The Lambda function performs the following tasks:

- reads Google Drive credentials from AWS Secrets Manager
- connects to the Google Drive folder containing healthcare datasets

- downloads CSV files
- uploads them to the S3 raw layer
- triggers the AWS Glue ETL job

Instead of manually invoking the Lambda function, a scheduled rule was created in **Amazon EventBridge** to run the pipeline automatically.

The scheduling architecture implemented in this project is shown below:

```
Amazon EventBridge (Schedule)
        ↓
AWS Lambda Ingestion Function
        ↓
Amazon S3 Raw Layer
        ↓
AWS Glue ETL Job
        ↓
Amazon S3 Processed Layer
```

The pipeline runs **twice per day** to simulate periodic healthcare data ingestion.

11.1 Open Amazon EventBridge

1. Log in to the **AWS Management Console**
2. Search for **EventBridge**
3. Open **Amazon EventBridge**

EventBridge is used in this project to run the ingestion pipeline automatically based on a scheduled trigger.

11.2 Create a New Schedule Rule

1. In the EventBridge console, click **Schedules**
2. Click **Create schedule**

Fill the following details.

Schedule name:

```
healthcare-ingestion-schedule
```

Description:

```
Triggers Lambda function to ingest healthcare datasets from Google Drive into
S3
```

Click **Next**

11.3 Configure the Schedule Pattern

This project requires the ingestion pipeline to run **twice per day**.

In the schedule configuration section:

Schedule type

```
Recurring schedule
```

Schedule pattern

```
Cron-based schedule
```

Cron expression used:

```
cron(0 5,17 * * ? *)
```

Meaning of this expression:

Field	Value	Meaning
Minutes	0	run at minute 0
Hours	5,17	run at 5 AM and 5 PM
Day of month	*	every day
Month	*	every month
Day of week	?	not specified
Year	*	every year

Therefore the pipeline runs:

- **5:00 AM**
- **5:00 PM**

every day.

Click **Next**

11.4 Configure Target Service

Next, configure the service that should run when the schedule triggers.

Target type

```
AWS service
```

Select service

```
Lambda
```

Select function

Choose the Lambda function created for ingestion.

Example:

```
healthcare_gdrive_ingestion_lambda
```

This function performs the following tasks:

- reads credentials from Secrets Manager
- connects to Google Drive
- downloads healthcare CSV files
- uploads them to S3 raw folders
- starts the Glue ETL job

Click **Next**

11.5 Configure Execution Role

EventBridge requires permission to invoke the Lambda function.

In the permissions section choose:

```
Create new role for this specific resource
```

AWS automatically creates a role that allows EventBridge to invoke the Lambda function.

Example role name created by AWS:

```
Amazon_EventBridge_Invoke_Lambda_healthcare
```

Click **Next**

11.6 Review and Create Schedule

Review the configuration:

Schedule name

```
healthcare-ingestion-schedule
```

Schedule type

```
Cron schedule
```

Cron expression

```
cron(0 5,17 * * ? *)
```

Target

```
healthcare_gdrive_ingestion_lambda
```

Click **Create schedule**

The EventBridge schedule is now active.

11.7 Verify the Schedule

After the schedule is created, verify that it appears in the EventBridge schedules list.

Steps:

1. Open **Amazon EventBridge**
2. Click **Schedules**
3. Confirm the schedule exists:

```
healthcare-ingestion-schedule
```

The status should show:

```
Enabled
```

11.8 Test the Scheduled Pipeline

To ensure the pipeline works correctly, the Lambda function can be tested manually before waiting for the scheduled run.

Step 1: Manually trigger Lambda

1. Open **AWS Lambda**
2. Open the ingestion function

```
healthcare_gdrive_ingestion_lambda
```

3. Click **Test**

Step 2: Verify ingestion process

After the Lambda function runs, verify that files are uploaded to S3.

Open:

```
S3 → healthcare-metrics-hmanthena → raw
```

You should see folders like:

```
raw/pbj/load_date=YYYY-MM-DD/
raw/provider/load_date=YYYY-MM-DD/
raw/quality/load_date=YYYY-MM-DD/
```

Step 3: Verify Glue job trigger

The Lambda function also starts the ETL job after ingestion completes.

Open:

```
AWS Glue → ETL Jobs → healthcare_etl_job
```

Check the **Runs tab** to verify that a new job run was triggered.

11.9 End-to-End Scheduled Execution Flow

Once the EventBridge rule is active, the pipeline executes automatically twice daily.

```
5:00 AM / 5:00 PM EventBridge Schedule
        ↓
Lambda Ingestion Function
        ↓
Read credentials from Secrets Manager
        ↓
Download healthcare datasets from Google Drive
        ↓
Upload files to S3 Raw Layer
        ↓
Trigger Glue ETL Job
        ↓
Transform data and calculate metrics
        ↓
Write Parquet output to S3 Processed Layer
        ↓
Data available for Athena and QuickSight dashboards
```

11.10 Outcome

By configuring Amazon EventBridge scheduling, the ingestion pipeline became fully automated.

This configuration ensures that:

- healthcare datasets are ingested regularly
- ETL jobs run only after ingestion completes
- manual intervention is not required
- the pipeline behaves like a production data workflow

This completes the **automated scheduling setup for the healthcare data pipeline.**

12. Glue Data Catalog (Raw Layer)

A Glue crawler scans the S3 raw layer and automatically creates tables.

Database

```
healthcare_raw_db
```

Crawler

```
healthcare-raw-crawler
```

Tables Generated

```
raw_pbj
raw_provider
raw_quality
```

The Glue Data Catalog allows downstream ETL jobs and SQL engines to access raw datasets.

13. AWS Glue ETL Pipeline

A **PySpark-based AWS Glue job** transforms the raw datasets.

Glue Job

```
healthcare_etl_job
```

Processing Steps

1. Read raw datasets from Glue Catalog
2. Clean and standardize data
3. Convert provider number to CCN
4. Calculate staffing metrics
5. Join provider data
6. Write curated data to S3

Metrics Calculated

Metric	Formula
Total Nurse Hours	hrs_rn + hrs_lpn + hrs_cna
Nurse-to-Patient Ratio	total_nurse_hours / mds_census
Bed Utilization Rate	mds_census / certified_beds

14. Processed Data Storage

Processed data is written in **Parquet format** for analytics efficiency.

Output Path

```
s3://healthcare-metrics-hmanthena/processed/fact_staffing_daily/
```

Partitioning

```
year_month=2024-04/
year_month=2024-05/
```

Parquet format improves:

- compression
- query performance
- cost efficiency

15. Processed Data Catalog

A second crawler creates catalog tables for processed datasets.

Database

```
healthcare_processed_db
```

Crawler

```
healthcare_processed_crawler
```

Tables

```
fact_staffing_daily
dim_facility
quality_measures
fact_staffing_monthly
```

16. Analytics Layer (Athena Views)

SQL views simplify analytics queries and dashboard consumption.

ed Fact View

```
v_fact_staffing_ed
```

Combines fact and dimension data.

Monthly KPI View

```
v_monthly_staffing_kpis
```

Includes:

- average nurse-to-patient ratio
- bed utilization rate
- total nurse hours
- patient census

Facility Ranking View

```
v_top_facilities_utilization
```

Used for leaderboard-style dashboards.

17. Monitoring and Observability Configuration

To monitor the healthcare ingestion and transformation pipeline, **Amazon CloudWatch** was configured for both the AWS Lambda ingestion function and the AWS Glue ETL job.

This setup was used to track:

- Lambda execution logs
- Glue ETL job logs
- ingestion failures
- ETL failures
- debugging details for pipeline issues

The monitoring flow in this project is:

```
EventBridge Schedule
        ↓
Lambda Ingestion Function
        ↓
CloudWatch Logs

Lambda starts Glue ETL Job
        ↓
Glue Job Execution
        ↓
CloudWatch Logs

Glue Job Failure / Timeout
        ↓
EventBridge Rule
        ↓
SNS Topic
        ↓
Email Alert
```

17.1 Configure CloudWatch Logs for the Lambda Ingestion Function

The Lambda function used in this project performs the following:

- reads Google Drive credentials from Secrets Manager
- connects to the Google Drive folder
- downloads source CSV files
- uploads files to S3 raw folders
- triggers the Glue ETL job

Since this is the first execution layer in the pipeline, logging was enabled here first.

Step 1: Open the Lambda function

1. Log in to the **AWS Management Console**
2. Search for **Lambda**
3. Open **AWS Lambda**
4. In the Functions list, open the ingestion Lambda function used for the project

This is the function that pulls files from Google Drive and loads them into:

```
s3://healthcare-metrics-hmanthena/raw/
```

Step 2: Verify the Lambda execution role

1. Inside the Lambda function page, open the **Configuration** tab
2. Click **Permissions**
3. Under **Execution role**, open the IAM role attached to the Lambda function

The Lambda execution role must include CloudWatch logging permissions. AWS documents that the managed policy `AWSLambdaBasicExecutionRole` grants the permissions required for uploading logs to CloudWatch.

Step 3: Attach Lambda logging permissions

In the IAM role attached to Lambda:

1. Click **Add permissions**
2. Choose **Attach policies**
3. Search for:

```
AWSLambdaBasicExecutionRole
```

4. Attach the policy

This policy allows Lambda to:

- create log groups
- create log streams
- push execution logs into CloudWatch

Step 4: Save and test the Lambda function

1. Go back to the Lambda function page
2. Click **Test**
3. Trigger the function manually once

After the function runs, Lambda automatically creates a CloudWatch log group if one does not already exist. AWS documents that Lambda sends function logs to CloudWatch Logs and you can view them from the Lambda console or directly in CloudWatch.

Step 5: View Lambda logs in CloudWatch

1. Open **CloudWatch**
2. In the left menu, click **Log groups**
3. Search for the Lambda log group in this format:

```
/aws/lambda/<your-lambda-function-name>
```

For your project, the log group will be for the ingestion Lambda function you created.

4. Open the latest log stream

Inside the logs, verify these events:

- Lambda started successfully
- secret was read from Secrets Manager
- Google Drive authentication succeeded
- source files were identified
- files uploaded successfully into S3 raw folders
- Glue job was triggered successfully

17.2 What was logged in the Lambda function

For this project, the Lambda logs should capture each key ingestion step so the full Google Drive to S3 flow can be debugged.

The function should log:

- start of ingestion run
- source folder access
- file name being processed
- S3 target path
- upload success message
- Glue job trigger response
- exception details if any step fails

Typical checkpoints logged in this project:

```
Starting scheduled ingestion
Reading Google Drive credentials from Secrets Manager
Authenticating to Google Drive
Found file: PBJ_Daily_Nurse_Staffing_Q2_2024.csv
Uploading file to raw/pbj/load_date=YYYY-MM-DD/
Found file: NH_ProviderInfo_Oct2024.csv
Uploading file to raw/provider/load_date=YYYY-MM-DD/
Found file: NH_QualityMsr_Claims_Oct2024.csv
Uploading file to raw/quality/load_date=YYYY-MM-DD/
All files uploaded successfully
Starting Glue job: healthcare_etl_job
```

These logs make it easy to identify exactly where the pipeline failed if ingestion breaks.

17.3 Configure CloudWatch Logs for the Glue ETL Job

After ingestion completes, Lambda starts the Glue ETL job:

```
healthcare_etl_job
```

This Glue job reads the raw healthcare datasets, applies transformations, calculates staffing metrics, and writes Parquet output into the processed layer.

AWS Glue provides real-time logging to CloudWatch Logs, and AWS documents that Glue job logs can be viewed through CloudWatch log groups.

Step 1: Open the Glue ETL job

1. Log in to the **AWS Management Console**
2. Search for **AWS Glue**
3. Open **AWS Glue**
4. In the left navigation, click **ETL jobs**
5. Open the job:

```
healthcare_etl_job
```

Step 2: Verify the IAM role used by Glue

Inside the job details page:

1. Confirm the job is using the correct IAM role
   Example from your setup:

```
AWSGlueServiceRole-healthcare
```

This role must have access to:

- read from S3 raw
- write to S3 processed
- access Glue Data Catalog
- publish logs to CloudWatch

Step 3: Check Glue logging settings

AWS states that Glue jobs support real-time logging in CloudWatch, and in Glue 5.0 all jobs have real-time logging available. AWS also documents optional parameters for explicit continuous logging configuration on older or customized setups.

For this project, verify the job is configured normally and then run it once. If explicit logging parameters are needed, add them under **Job details → Advanced properties → Job parameters**.

Use:

```
--enable-continuous-cloudwatch-log=true
--continuous-log-logGroup=/aws-glue/jobs/logs-v2
--continuous-log-logStreamPrefix=healthcare-etl
```

These settings make Glue job logs easier to find in CloudWatch.

Step 4: Run the Glue job

1. In the Glue job page, click **Run**
2. Wait for the job to start
3. Note the **Job run ID**

This run ID helps locate the exact log stream later.

Step 5: View Glue logs in CloudWatch

1. Open **CloudWatch**
2. Click **Log groups**
3. Search for Glue log groups such as:

```
/aws-glue/jobs/error
/aws-glue/jobs/output
/aws-glue/jobs/logs-v2
```

AWS documents these log groups for viewing Glue job logs in CloudWatch.

4. Open the relevant log group
5. Search by the Glue **job run ID**

17.4 What was verified in Glue logs for this project

In this healthcare pipeline, Glue logs were used to verify:

- raw tables were read successfully from `healthcare_raw_db`
- PBJ, Provider, and Quality datasets loaded properly
- `PROVNUM` was padded and converted to `CCN`
- `work_date` conversion completed
- `total_nurse_hours` was calculated
- `nurse_to_patient_ratio` was calculated
- joins between PBJ and Provider completed successfully
- processed Parquet files were written into:

```
s3://healthcare-metrics-hmanthena/processed/
```

- partition folders such as `year_month=2024-04` were created successfully

If the ETL failed, the CloudWatch logs were used to identify the exact error line and Spark exception.

AWS also documents Glue job run insights for failure analysis when enabled.

17.5 Common issues debugged through CloudWatch in this project

Lambda-side issues

CloudWatch Lambda logs help detect:

- invalid or malformed Google service account JSON
- Secrets Manager access failure
- Google Drive authentication issue
- missing file in Google Drive folder
- Google Drive folder not shared with service account
- S3 upload permission error
- Lambda timeout
- `glue:StartJobRun` permission missing

Glue-side issues

CloudWatch Glue logs help detect:

- crawler table not found in `healthcare_raw_db`
- incorrect column names in source CSV
- data type mismatch during transformations
- join key mismatch on `ccn`
- null or zero division issues in metric calculation
- failure writing Parquet output to processed S3 path
- partition write errors

17.6 Configure failure alerts for the Glue job

For this project, Glue job failures were monitored using **EventBridge + SNS**.

AWS documents that EventBridge rules can be created using AWS service event patterns, and Glue job state changes can be used to trigger notifications.

This was configured so that if the Glue job enters:

- `FAILED`
- `TIMEOUT`

an email alert is sent automatically.

Step 1: Create an SNS topic

1. Open **AWS Console**
2. Search for **SNS**
3. Open **Amazon SNS**
4. Click **Topics**
5. Click **Create topic**
6. Choose **Standard**

7. Topic name:

```
healthcare-glue-failure-alerts
```

8. Click **Create topic**

Step 2: Add email subscription

1. Open the newly created SNS topic
2. Click **Create subscription**
3. Protocol: **Email**
4. Endpoint: enter your email address
5. Click **Create subscription**
6. Open your email inbox
7. Confirm the SNS subscription

Without email confirmation, alerts will not be delivered.

Step 3: Create an EventBridge rule for Glue failures

1. Open **Amazon EventBridge**
2. Click **Rules**
3. Click **Create rule**
4. Rule name:

```
glue-job-failure-rule
```

5. Event bus: **default**
6. Rule type: **Rule with an event pattern**
7. Click **Next**

AWS documents the console flow for creating EventBridge rules based on event patterns.

Step 4: Define the event pattern

In the event pattern section, configure:

- **Event source**: AWS events or EventBridge partner events
- **AWS service**: Glue
- **Event type**: Glue Job State Change

Set the filter to monitor:

- job name = `healthcare_etl_job`
- state = `FAILED`
- state = `TIMEOUT`

This ensures alerts are only triggered for your ETL job.

Step 5: Add SNS as the target

1. In the target section, choose **SNS topic**
2. Select:

```
healthcare-glue-failure-alerts
```

3. Complete rule creation

Now when the job fails, EventBridge sends the event to SNS, and SNS sends the email notification.

17.7 Final observability setup used in this project

The monitoring configuration for this healthcare architecture was completed as follows:

Lambda logging

- Lambda execution role was given `AWSLambdaBasicExecutionRole`
- ingestion Lambda logs were sent to CloudWatch
- logs were reviewed under `/aws/lambda/<lambda-function-name>`

Glue logging

- Glue job `healthcare_etl_job` was run with CloudWatch logging enabled
- ETL logs were reviewed in Glue-related CloudWatch log groups
- job run ID was used to trace specific runs

Failure alerting

- SNS topic `healthcare-glue-failure-alerts` was created
- email subscription was confirmed
- EventBridge rule `glue-job-failure-rule` was created
- alerts were configured for `FAILED` and `TIMEOUT` states of `healthcare_etl_job`

17.8 Outcome

With this setup, the project achieved full monitoring coverage for both ingestion and transformation.

This allowed the pipeline to support:

- operational visibility
- faster debugging
- failure traceability
- production-style alerting
- better support for scheduled runs

18. Failure Alerts

Glue job failures trigger automated alerts.

Failure Events

- FAILED
- TIMEOUT

Alert Flow

```
Glue Job Failure
      |
      ▼
EventBridge Rule
      |
      ▼
SNS Notification
      |
      ▼
Email / Slack Alert
```

This enables **quick operational response.**

Glue Job Failure Alerts Configuration (EventBridge + SNS)

To ensure operational reliability of the healthcare data pipeline, automated alerts were configured for AWS Glue job failures.

The ETL job in this project is:

```
healthcare_etl_job
```

If the Glue job fails or times out during execution, an alert notification is automatically sent via **Amazon SNS**. This allows the data engineer to quickly detect and investigate pipeline failures.

The alerting workflow implemented in this project is:

```
Glue ETL Job Execution
        ↓
Glue Job State Change Event
        ↓
Amazon EventBridge Rule
        ↓
Amazon SNS Topic
        ↓
Email Notification
```

This ensures that any ETL failures are immediately visible without manually checking job logs.

18.1 Create SNS Topic for Failure Alerts

Amazon SNS is used to deliver notifications when the Glue job fails.

Step 1: Open Amazon SNS

1. Log in to the **AWS Management Console**
2. Search for **SNS**
3. Open **Amazon Simple Notification Service (SNS)**

Step 2: Create a new topic

1. Click **Topics**
2. Click **Create topic**
3. Choose **Standard**

Fill the following details:

Topic name:

```
healthcare-glue-failure-alerts
```

Display name (optional):

```
Healthcare Glue Alerts
```

4. Click **Create topic**

This topic will be used to send failure notifications.

18.2 Create Email Subscription

The SNS topic must have a subscription so that alerts are delivered to an email address.

Step 1: Open the topic

1. In SNS → Topics
2. Select:

```
healthcare-glue-failure-alerts
```

Step 2: Add subscription

1. Click **Create subscription**

Fill:

Protocol

```
Email
```

Endpoint

```
your-email-address
```

Example:

```
harikha@email.com
```

2. Click **Create subscription**

Step 3: Confirm email subscription

1. Open your email inbox
2. Find the **SNS confirmation email**
3. Click **Confirm subscription**

After confirmation, the SNS topic can send alert notifications.

18.3 Create EventBridge Rule for Glue Job Failures

Amazon EventBridge is used to detect when the Glue ETL job fails.

Step 1: Open Amazon EventBridge

1. Log in to AWS Console
2. Search for **EventBridge**
3. Open **Amazon EventBridge**

Step 2: Create a rule

1. Click **Rules**
2. Click **Create rule**

Fill:

Rule name

```
healthcare-glue-job-failure-rule
```

Description

```
Trigger alert when healthcare ETL job fails
```

Event bus

```
default
```

Rule type

```
Rule with an event pattern
```

Click **Next**

18.4 Define Event Pattern for Glue Failures

EventBridge listens for Glue job state change events.

Configure the event pattern:

Event source

```
AWS events or EventBridge partner events
```

AWS service

```
Glue
```

Event type

```
Glue Job State Change
```

Filter the specific job

To ensure alerts are only triggered for the ETL job used in this project, apply filters.

Job name

```
healthcare_etl_job
```

Job state

Select:

```
FAILED
TIMEOUT
```

This ensures alerts trigger only when:

- the Glue job fails
- the Glue job times out

Click **Next**

18.5 Configure Target (SNS Topic)

Now configure the SNS topic as the alert destination.

Step 1: Select target

Target type

```
AWS service
```

Select target

```
SNS topic
```

Step 2: Choose topic

Select the topic created earlier:

```
healthcare-glue-failure-alerts
```

Click **Next**

18.6 Review and Create Rule

1. Review rule configuration
2. Click **Create rule**

The EventBridge rule is now active.

18.7 Test the Alert Configuration

To verify the alert system works correctly, the Glue job can be triggered manually.

Step 1: Run the Glue job

1. Open **AWS Glue**
2. Go to **ETL Jobs**
3. Open:

```
healthcare_etl_job
```

4. Click **Run**

18.8 Alert Notification

When the Glue job fails, the email notification sent by SNS contains information similar to:

```
Glue Job Failed

Job Name: healthcare_etl_job
State: FAILED
Time: 2026-03-05T17:05:10Z
Region: us-east-1
```

This helps quickly identify the failed pipeline run.

18.9 Operational Benefits

This alert configuration provides several benefits:

Immediate failure detection

Pipeline failures are reported automatically.

Faster troubleshooting

Engineers can immediately open:

- CloudWatch logs
- Glue job logs
- Lambda logs

to identify the issue.

Production-style monitoring

This setup reflects how real enterprise pipelines monitor ETL workflows.

18.10 Final Monitoring Architecture

The final monitoring architecture implemented in this project is:

```
EventBridge Schedule
        ↓
Lambda Ingestion
        ↓
CloudWatch Logs

Lambda triggers Glue ETL
        ↓
Glue Job Execution
        ↓
CloudWatch Logs

Glue Job Failure
        ↓
```

```
EventBridge Rule
        ↓
SNS Topic
        ↓
Email Alert
```

This completes the **monitoring and alerting framework** for the healthcare analytics pipeline.

19. Dashboard & Analytics

An **Amazon QuickSight dashboard** provides operational insights.

Key Visualizations

1️⃣ Average Nurse-to-Patient Ratio Trend
2️⃣ Nurse Staffing Distribution Across Facilities
3️⃣ Average Nurse-to-Patient Ratio by Provider
4️⃣ Total Nurse Hours by Hospital
5️⃣ Patient Census Trends

These insights help healthcare administrators monitor staffing efficiency and facility utilization.

20. Data Validation and Analytics Queries

After the AWS Glue ETL job processes the raw healthcare datasets, the transformed tables are registered in the **AWS Glue Data Catalog** and can be queried using **Amazon Athena**.

These queries are used for:

- validating processed datasets
- generating analytics metrics
- powering dashboard visualizations

20.1 Validate Processed Tables

Verify that the transformed tables were created successfully.

```
SELECT *
FROM healthcare_processed_db.fact_staffing_daily
LIMIT 10;
```

20.2 Monthly Staffing Trend Analysis

This query calculates the **average nurse-to-patient ratio trend over time**, which is used in the dashboard trend chart.

```
SELECT
    year_month,
    AVG(nurse_to_patient_ratio) AS avg_ratio
FROM healthcare_processed_db.fact_staffing_daily
GROUP BY year_month
ORDER BY year_month;
```

This query powers the visualization:

Trend Chart – Average Nurse-to-Patient Ratio Trend

20.3 State-Level Staffing Analysis

This query analyzes staffing levels across different states.

```
SELECT
    d.facility_state,
    AVG(f.nurse_to_patient_ratio) AS avg_ratio
FROM healthcare_processed_db.fact_staffing_daily f
JOIN healthcare_processed_db.dim_facility d
    ON f.ccn = d.ccn
GROUP BY d.facility_state
ORDER BY avg_ratio DESC;
```

This helps identify:

- states with **better staffing coverage**
- regions with **higher workload pressure**

20.4 Facility Utilization KPI

This query calculates the **average bed utilization rate by hospital**.

```
SELECT
    ccn,
    AVG(bed_utilization_rate) AS avg_utilization
FROM healthcare_processed_db.fact_staffing_daily
GROUP BY ccn
ORDER BY avg_utilization DESC
LIMIT 10;
```

Used for identifying:

- hospitals operating near capacity
- facilities with lower utilization

20.5 Top Hospitals by Patient Census

This query identifies hospitals with the highest patient volume.

```
SELECT
    provider_name,
    SUM(mds_census) AS total_patients
FROM healthcare_processed_db.v_fact_staffing_ed
GROUP BY provider_name
ORDER BY total_patients DESC
LIMIT 10;
```

Used for:

Top 10 hospitals with highest patient throughput

21. Analytical Views

To simplify reporting and dashboard development, several **Athena SQL views** are created.

21.1 ed Fact View

This view joins fact and dimension tables to provide a single dataset for analytics.

```
CREATE VIEW healthcare_processed_db.v_fact_staffing_ed AS
SELECT
    f.ccn,
    f.work_date,
    f.year_month,
```

```
    d.facility_state,
    d."provider name" AS provider_name,
    f.mds_census,
    f.hrs_rn,
    f.hrs_lpn,
    f.hrs_cna,
    f.total_nurse_hours,
    f.nurse_to_patient_ratio,
    f.bed_utilization_rate
FROM healthcare_processed_db.fact_staffing_daily f
LEFT JOIN healthcare_processed_db.dim_facility d
ON f.ccn = d.ccn;
```

This simplifies queries by eliminating the need for repeated joins.

21.2 Monthly KPI View

This view aggregates monthly healthcare staffing KPIs.

```
CREATE OR REPLACE VIEW healthcare_processed_db.v_monthly_staffing_kpis AS
SELECT
    year_month,
    AVG(nurse_to_patient_ratio) AS avg_nurse_to_patient_ratio,
    AVG(bed_utilization_rate) AS avg_bed_utilization_rate,
    SUM(total_nurse_hours) AS total_nurse_hours,
    AVG(mds_census) AS avg_mds_census
FROM healthcare_processed_db.v_fact_staffing_ed
GROUP BY year_month
ORDER BY year_month;
```

Used for:

- trend charts
- operational KPI tracking

21.3 Top Facility Utilization View

This view identifies top performing healthcare facilities.

```
CREATE OR REPLACE VIEW healthcare_processed_db.v_top_facilities_utilization
AS
SELECT
    provider_name,
    ccn,
    AVG(bed_utilization_rate) AS avg_bed_utilization_rate,
    AVG(nurse_to_patient_ratio) AS avg_nurse_to_patient_ratio,
    SUM(total_nurse_hours) AS total_nurse_hours
```

```
FROM healthcare_processed_db.v_fact_staffing_ed
GROUP BY provider_name, ccn
ORDER BY avg_bed_utilization_rate DESC;
```

This supports:

- facility performance comparison
- operational benchmarking

25. Dashboard Metrics Powered by SQL

The following metrics are generated using the SQL queries and views.

Staffing Metrics

- Average nurse-to-patient ratio
- Total nurse staffing hours
- Staffing trend over time

Facility Metrics

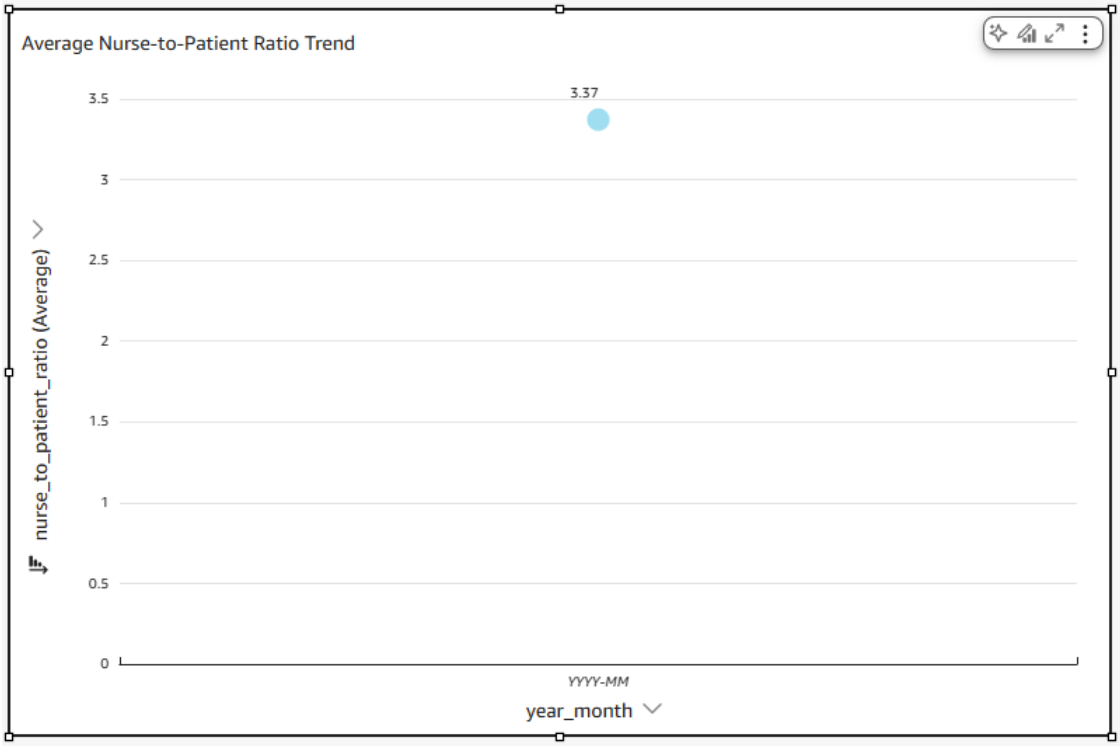
- Bed utilization rate
- Occupancy trends
- Staffing vs patient load comparison
- Top hospitals by patient census

These metrics are visualized in the **Amazon QuickSight dashboard**.

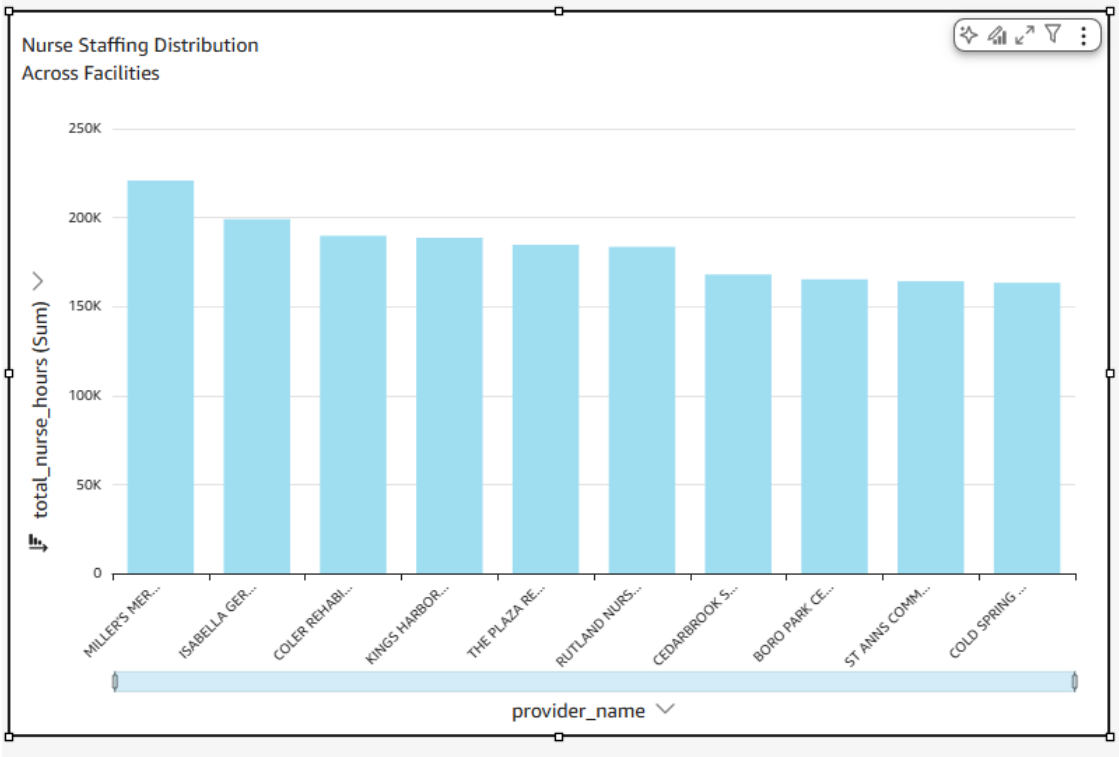
Please find below metrics screnshots---

1. **Key Metic- Average Nurse-to-Patient Ratio Trend KPI**

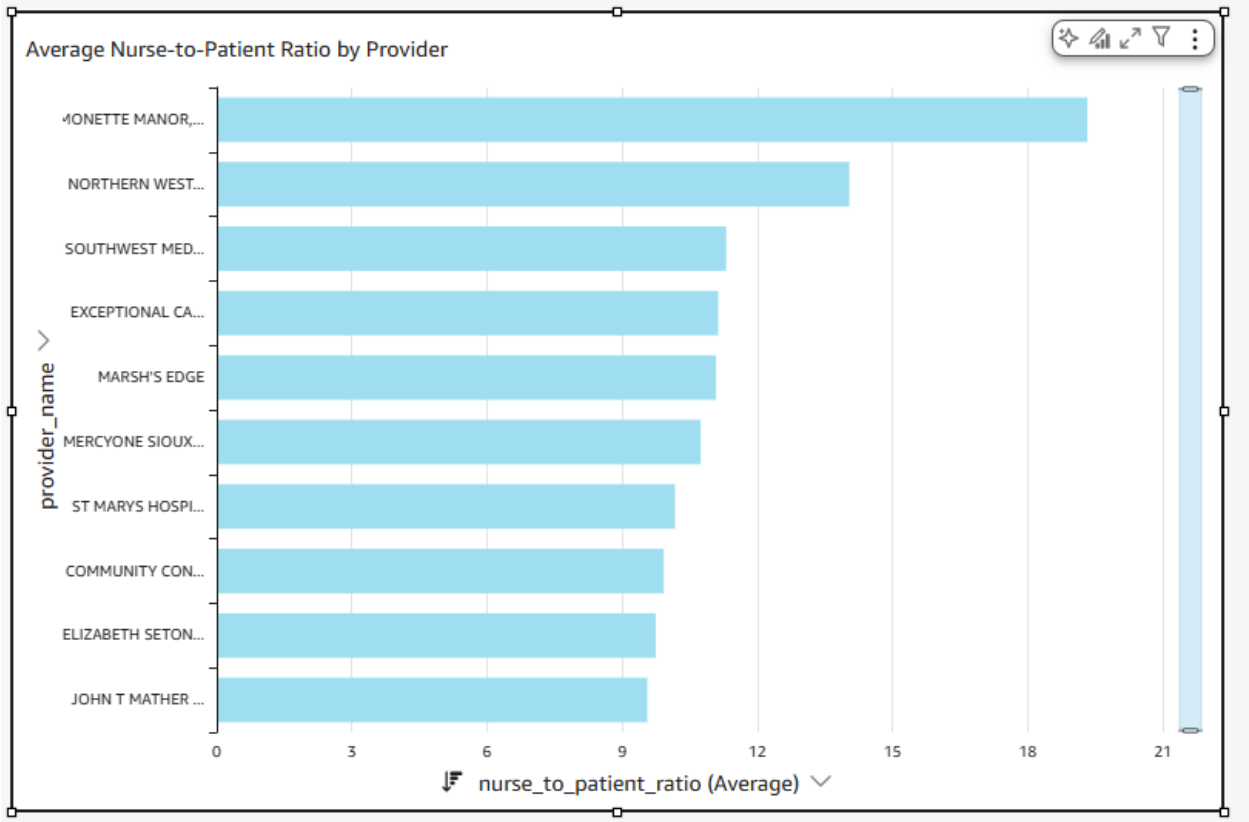
2. **Trend Chart- Average Nurse-to-Patient Ratio Trend**



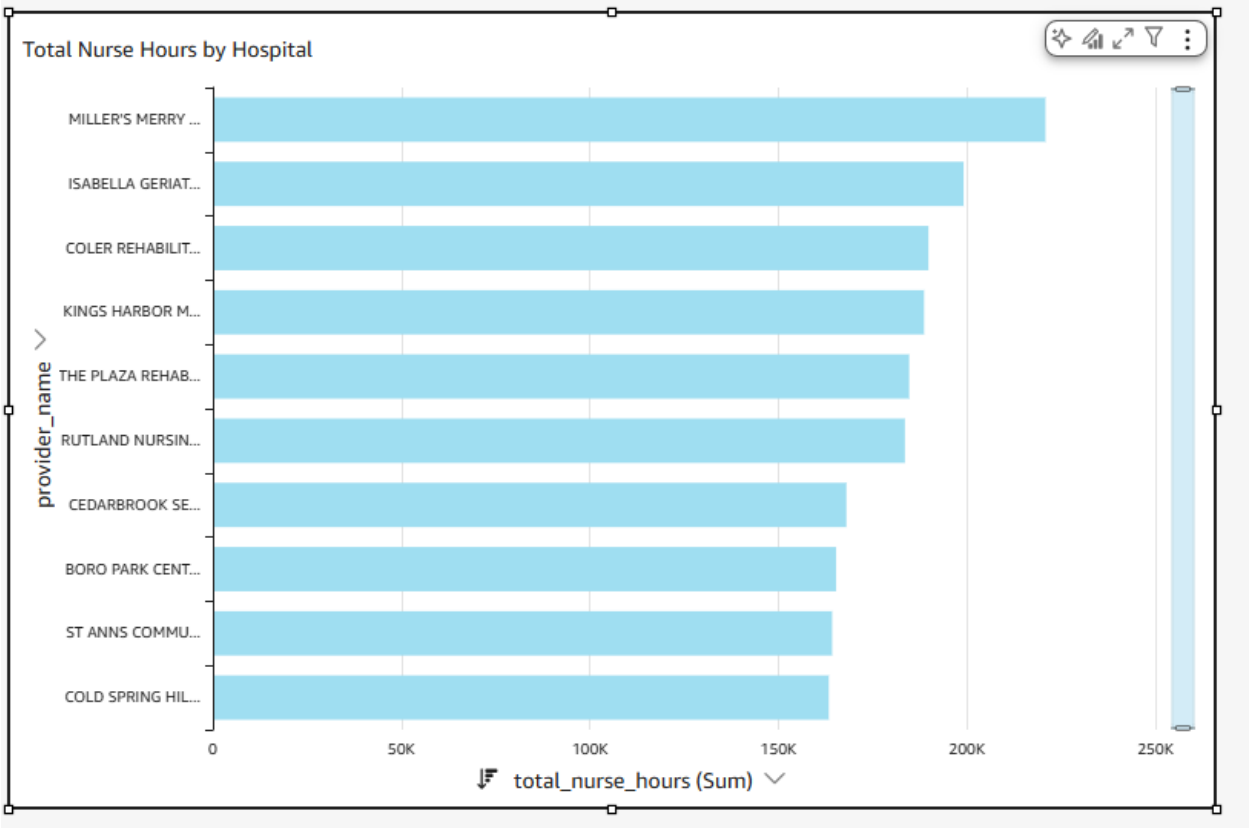
3. **Staffing comparision bar chart- Nurse Staffing Distribution Across Facilities**



4. **Provider Comparision – Average Nurse-to-Patient Ratio by Provider**



5. **Hospital workload – Total Nurse Hours by Hospital**



26. Testing

How I Tested the Lambda Ingestion Flow

1. Initial Pipeline Run (First Execution)

I first tested the Lambda function by invoking it manually from the AWS Lambda console.

Steps performed

1. Deployed the Lambda code.
2. Created a **test event** with an empty payload `{}`.
3. Executed the Lambda function.

What happened

- Lambda authenticated with Google Drive using the service account stored in **Secrets Manager**.
- It listed files from the **Healthcare_datasets** folder.
- It downloaded the three expected CSV files.

- Each file was uploaded into the correct S3 raw path.

Resulting S3 structure

Example:

```
s3://healthcare-metrics-hmanthena
    raw/
       pbj/
           load_date=2026-03-09/
               PBJ_Daily_Nurse_Staffing_Q2_2024.csv
       provider/
           load_date=2026-03-09/
               NH_ProviderInfo_Oct2024.csv
       quality/
           load_date=2026-03-09/
               NH_QualityMsr_Claims_Oct2024.csv
```

This confirmed the **Google Drive → Lambda → S3 ingestion path was working correctly**.

2. Logging Validation

I verified that logs were generated correctly.

Where I checked logs

```
AWS Console
→ CloudWatch
→ Log groups
→ /aws/lambda/gdrive_to_s3_healthcare_loader
```

Sample logs observed

```
INFO Lambda ingestion started
INFO Fetching Google service account secret from Secrets Manager
INFO Building Google Drive service
INFO Listing files from Google Drive
INFO Found 3 files in Google Drive folder

INFO Processing file PBJ_Daily_Nurse_Staffing_Q2_2024.csv
INFO Download completed
INFO Uploaded to s3://healthcare-metrics-hmanthena/raw/pbj/load_date=2026-03-
09/

INFO Processing file NH_ProviderInfo_Oct2024.csv
INFO Upload completed

INFO Triggering Glue job healthcare_etl_job
INFO Lambda ingestion completed successfully
```

This validated that **structured logging is working correctly** and logs are available for monitoring.

3. Incremental Run Testing

To verify incremental ingestion, I executed the Lambda function again **without modifying the source files**.

Expected behavior

Files that were already processed should be skipped.

Observed behavior

The logs showed:

```
INFO Skipping unchanged file: PBJ_Daily_Nurse_Staffing_Q2_2024.csv
INFO Skipping unchanged file: NH_ProviderInfo_Oct2024.csv
INFO Skipping unchanged file: NH_QualityMsr_Claims_Oct2024.csv
INFO No new files uploaded. Glue job will not be triggered.
```

This confirmed the **checkpoint logic is working** and prevents duplicate ingestion.

4. Testing New File Detection

To validate incremental processing further:

1. I updated one file in the Google Drive folder.
2. Ran the Lambda function again.

Observed behavior

```
INFO New or updated file detected: PBJ_Daily_Nurse_Staffing_Q2_2024.csv
INFO Downloading file
INFO Upload completed
INFO Triggering Glue job
```

Only the updated file was processed.

5. Glue Job Trigger Validation

I also verified that Glue is triggered only when new files are ingested.

Observed behavior

When new files were uploaded:

```
INFO Triggering Glue job healthcare_etl_job
INFO Glue job started successfully
```

When no new files were detected:

```
INFO No new files uploaded. Glue job will not be triggered.
```

This confirmed **downstream processing is controlled properly**.

6. Exception Handling Testing

I simulated failures to confirm error handling.

Example scenarios tested:

Scenario 1: Wrong S3 bucket

Lambda logged:

```
ERROR Failed to upload file to S3
Traceback ...
```

Scenario 2: Invalid Google credentials

Lambda logged:

```
ERROR Failed to create Google Drive service
Traceback ...
```

These errors appeared correctly in **CloudWatch logs**, confirming that exception handling is functioning.

7. Scheduled Execution Validation

Finally, I verified the scheduled pipeline.

EventBridge Schedule

```
cron(0 5,17 * * ? *)
```

This triggers Lambda:

- 5 AM
- 5 PM

End-to-end automated flow

```
EventBridge
     ↓
Lambda ingestion
     ↓
S3 raw layer
     ↓
Glue ETL job
```

27. Final Architecture Layers

The system follows a **three-layer data architecture**.

Raw Layer

```
Google Drive → Lambda → S3 Raw
```

Processed Layer

```
Glue ETL → Parquet → S3 Processed
```

Analytics Layer

```
Athena Views → QuickSight Dashboard
```

28. Technologies Used

Category	Technology
Storage	Amazon S3
Ingestion	AWS Lambda
Scheduling	Amazon EventBridge
ETL	AWS Glue (PySpark)
Catalog	AWS Glue Data Catalog
Query Engine	Amazon Athena
Visualization	Amazon QuickSight
Monitoring	Amazon CloudWatch
Alerting	Amazon SNS
Source	Google Drive

29. Key Outcomes

This architecture successfully implements:

Data Engineering Capabilities

- Automated ingestion pipeline
- Serverless ETL processing
- Partitioned data lake architecture
- Schema discovery with Glue
- Cloud-native monitoring and alerting

Healthcare Analytics

- Nurse-to-patient ratio tracking
- Nurse staffing hours analysis
- Bed utilization monitoring
- Hospital performance comparison